

HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN

Báo cáo Bài tập lớn — Môn Lập trình nâng cao
Ứng dụng desktop Client–Server (Java 21, JavaFX, SQLite)

1. Mục tiêu và phạm vi thực hiện

Mục tiêu: Xây dựng một sàn đấu giá trực tuyến nhiều người dùng theo mô hình Client–Server, cho phép nhiều người tham gia đặt giá đồng thời theo thời gian thực, hệ thống tự động quản lý vòng đời phiên đấu giá và quyết toán tài chính một cách an toàn trong môi trường đa luồng.

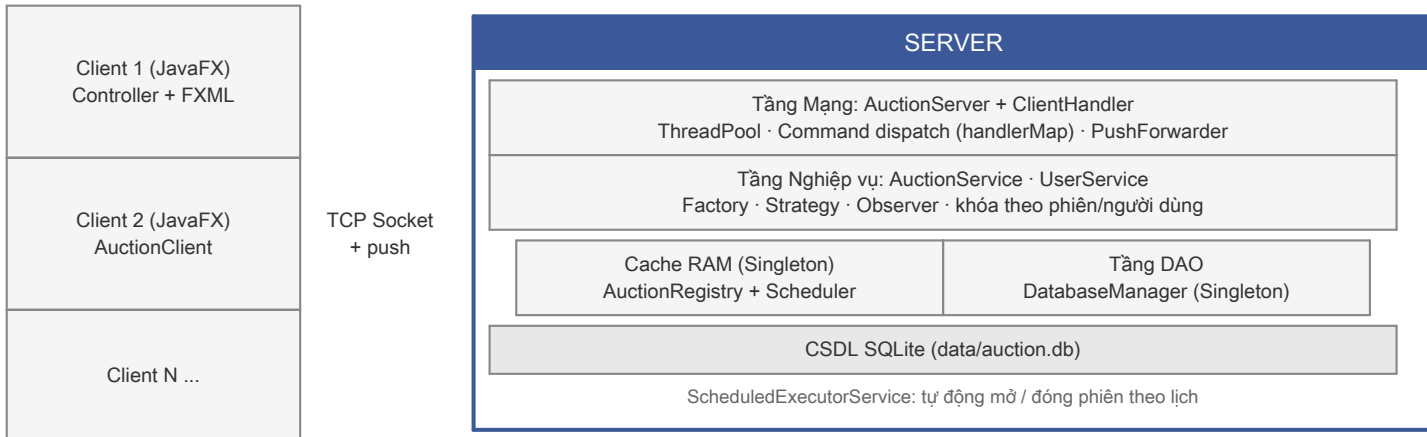
Phạm vi:

- Xác thực và phân quyền ba vai trò: Người mua (Bidder), Người bán (Seller), Quản trị (Admin).
- Quản lý sản phẩm theo nhóm và quản lý phiên đấu giá (tạo, hủy, mở/đóng theo lịch).
- Đặt giá thủ công, đấu giá tự động (proxy bidding), chống bắn giá phút chót (anti-sniping).
- Đẩy cập nhật giá tức thời tới nhiều client qua cơ chế Observer trên nền socket TCP.
- Quyết toán khi kết thúc: trừ số dư người thắng, cộng doanh thu người bán.
- Lưu trữ bền vững bằng SQLite kết hợp cache trên RAM.

Hệ thống là ứng dụng desktop (JavaFX), client bắt buộc kết nối được Server mới hoạt động (không có chế độ offline).

2. Kiến trúc tổng thể của hệ thống

Hệ thống gồm nhiều Client kết nối tới một Server trung tâm qua socket TCP (cổng mặc định 9999), trao đổi đối tượng Message bằng Java Object Serialization. Bên trong Server, kiến trúc được phân tầng rõ ràng: tầng mạng → tầng nghiệp vụ (Service) → tầng truy cập dữ liệu (DAO) → CSDL SQLite, có lớp cache trên RAM tăng tốc đọc.



Lưuồng xử lý một yêu cầu: Client gửi Message → ClientHandler (mỗi client một luồng) đọc và tra handlerMap để chọn handler tương ứng (Command-style dispatch) → gọi Service xử lý nghiệp vụ → Service đọc/ghi qua Cache và DAO. Khi trạng thái phiên thay đổi (giá mới, mở/đóng phiên), AuctionEventDispatcher phát sự kiện, PushForwarder (Observer) chuyển thành bản tin push và gửi tới mọi client đang theo dõi phiên đó.

3. Các chức năng đạt được (theo barem)

Chức năng	Hướng giải quyết	Lý do lựa chọn
Đăng ký / Đăng nhập, phân quyền	Validate dữ liệu, kiểm tra trùng username/email, băm mật khẩu trước khi lưu; tạo đối tượng User đúng vai trò bằng switch expression.	Không lưu mật khẩu thô để bảo mật; switch theo role giúp mở rộng vai trò dễ dàng, gọn gàng.

Chức năng	Hướng giải quyết	Lý do lựa chọn
Quản lý sản phẩm theo nhóm	Dùng Factory Method (ItemFactory + các Creator cho ART/ELECTRONICS/VEHICLE) để tạo sản phẩm theo nhóm.	Tách logic khởi tạo từng loại, thêm nhóm mới chỉ cần thêm Creator, không sửa code cũ (Open/Closed).
Tạo / hủy / kết thúc phiên đấu giá	Vòng đời trạng thái OPEN→RUNNING→FINISHED→PAID (nhánh CANCELED); ScheduledExecutorService tự mở và đóng phiên theo lịch.	State machine rõ ràng, dễ kiểm soát; scheduler tự động hóa, không cần thao tác thủ công đúng giờ.
Kiểm tra hợp lệ khi đặt giá	Dùng Strategy: StandardBidValidation và ReservePriceBidValidation (kiểm tra giá sàn).	Đối/đáp thêm luật đấu giá mà không sửa lỗi; mỗi luật là một lớp độc lập, dễ kiểm thử.
Đấu giá tự động (proxy bidding)	Người dùng đặt mức giá tối đa; hệ thống tự tăng giá tối thiểu cho người này khi bị vượt, xử lý theo thứ tự đăng ký (FIFO).	Mô phỏng cơ chế eBay quen thuộc, người dùng không phải canh giá liên tục.
Chống bán giá phút chót (anti-sniping)	Nếu có giá đặt trong ngưỡng 30s cuối, tự gia hạn thời gian đóng thành "hiện tại + 60s".	Đảm bảo công bằng, tránh thắng nhờ đặt giá sát giây cuối; logic đặt trong domain nên nhất quán.
Cập nhật real-time nhiều client	Observer: AuctionEventDispatcher phát sự kiện, PushForwarder đăng ký theo từng phiên và đẩy bản tin (NEW_BID, AUTO_BID, STARTED, ENDED, CANCELED).	Tách nguồn sự kiện khỏi nơi nhận; chỉ đẩy tới client quan tâm đúng phiên, tiết kiệm băng thông.
Quyết toán tài chính	Khi kết thúc: trừ số dư người thắng và cộng doanh thu người bán, mỗi thao tác dưới khóa theo từng người dùng.	Khóa theo người dùng tránh tranh chấp số dư khi nhiều giao dịch đồng thời mà không chặn chéo.
Lưu trữ bền vững + hiệu năng	SQLite qua DAO + cache RAM (cache-aside): đọc từ cache, thiếu thì nạp từ DB và làm ấm cache.	Giảm truy vấn DB lặp lại, vẫn đảm bảo dữ liệu bền vững khi tắt server.
Xử lý đồng thời nhiều client	Server đa luồng (ThreadPool), ConcurrentHashMap, khóa ReentrantLock theo từng phiên qua Guava Cache.	Khóa hạt mịn theo phiên/người dùng cho thông lượng cao mà vẫn an toàn dữ liệu.

Mẫu thiết kế & kỹ thuật đồng bộ áp dụng

Design Patterns: Singleton (AuctionRegistry, DatabaseManager, AuctionEventDispatcher), Factory Method (sản phẩm), Strategy (luật đặt giá), Observer (push real-time), Command-style dispatch (handlerMap). Đồng bộ: double-checked locking, ReentrantLock hạt mịn theo phiên/người dùng, ConcurrentHashMap/CopyOnWriteArrayList, ScheduledExecutorService cho phiên hẹn giờ.

4. Phân chia công việc giữa các thành viên

Thành viên	Công việc phụ trách
Trần Đăng Minh	Design Patterns (Factory, Singleton, Observer, Strategy), Exception
Nguyễn Đình Phúc	DAO, Database (SQLite), Unit Test
Nguyễn Ngọc Triệu	Service layer, Network layer (AuctionServer, ClientHandler, Push system)
Trịnh Tuấn Nghĩa	Controller, Giao diện JavaFX (FXML + CSS)

Trần Đăng Minh & Trịnh Tuấn Nghĩa cùng phối hợp phần Model (Entity, User, Item, Auction, Transaction) do khối lượng lớn.